A FAIR and federated repository prototype for multimodal Home-Cage Monitoring biomedical data

Damien Huzard[1,2*], Antoine Bluchet[3*], Kevin Dunglas[3]

* Damien and Antoine contributed equally to the work

Corresponding authors:
damien@metadatapp.net, antoine@les-tilleuls.coop, kevin@les-tilleuls.coop

Affiliations:

[1] Metadatapp, Montpellier, France
[2] NeuroNautix, Montpellier, France
[3] Les-Tilleuls.coop, Lille, France

Keywords:

Home-Cage Monitoring (HCM), FAIR Principles, Federated Repository, Zero-Copy Federation, Solid Pods, Data Plane, Control Plane

Abstract

Home-Cage Monitoring (HCM) produces vast, multimodal datasets whose reuse is critically limited by data fragmentation, proprietary formats, and inconsistent metadata. To address this reproducibility bottleneck while respecting institutional data governance, we present a working prototype of a novel, decoupled, federated repository architecture aligned with the FAIR principles. The core solution separates a local Data Plane (Solid Pods), responsible for secure storage and storage-level access control (WAC/ACP), from a Control Plane (Node Control Service via API Platform), which manages validation (MNMS and SHACL shapes) and metadata exposure. Federation is achieved through DCAT-compliant catalogue endpoints that enable Zero-Copy Federation: only standardized metadata is harvested by a central portal for global discovery, ensuring raw data always remains under the originating institution's control. This architecture provides a scalable framework for validated data ingestion, semantic interoperability, and cross-site search, transforming multimodal HCM data into a reusable community asset.

1. Introduction

Home-Cage Monitoring systems (HCM) produce the kind of dense, longitudinal behavioural record that many labs want but few can reuse. These systems run

continuously and can capture behaviour in context across the experimental timeline (Bains et al., 2018; Kahnau et al., 2023; Richardson, 2015). In practice, however, datasets end up locked inside local conventions: proprietary exports, lab-specific folder structures, missing environmental context (e.g., temperature, light cycle), and weak links between video and the time-series it should interpret. That makes cross-study synthesis brittle and slows down reproducibility efforts.

FAIR principles—Findable, Accessible, Interoperable, Reusable—describe the properties data should have to support discovery and reuse at scale (Wilkinson et al., 2016). For HCM, "FAIR" is not an abstract label: it translates into concrete requirements such as validated ingestion, stable identifiers, machine-readable catalogues, and access control that survives beyond a single web app.

At the same time, a centralised repository model is often a non-starter for institutions. HCM datasets can be large, sensitive, and entangled with governance constraints. The practical tension is familiar: global discoverability is valuable, but institutions need control over custody, permissions, and infrastructure.

In this article, we describe a prototype repository that treats that tension as a design constraint rather than a policy problem. The core move is architectural decoupling: a local Data Plane for storage and enforcement of access policies, and a Control Plane for validation, governance, and metadata exposure. Federation is achieved through DCAT catalogue harvesting, enabling "zero-copy" discovery without centralising raw data. We summarise why existing repository solutions fall short for HCM, define prototype-driven requirements, and present an implementation and adoption pathway toward a community-scale resource.

2. Requirements and Justification

Members from the COST action TEATIME (CA20135) conducted a structured assessment of repository solutions to determine whether an existing platform could support the realities of HCM data. That assessment is reported in a preprint (Bains et al., 2025) and focused on systems that claim support for behavioural, physiological, or imaging data, evaluating support for rich experimental metadata, continuous time-series, video assets, programmatic ingestion, and federation under institutional governance constraints. Across candidates, the same gaps recurred: weak linkage between video and aligned signals, limited machine-actionable validation at ingestion, and federation mechanisms that do not align well with institutional data sovereignty. The evaluated systems and screening criteria are documented in the preprint (Bains et al., 2025); here, we focus on a specific architecture and its prototype implementation.

In short, the common "upload supplementary files" model does not fit the HCM field. HCM needs explicit multimodal linkage, consistent contextual metadata, and interfaces that support automated workflows rather than manual curation after the fact. These requirements motivate a dedicated specification and a tailored repository framework rather than another general-purpose file store with a metadata form.

2.1 Prototype-Driven Design Requirements

To address the identified limitations while ensuring full compliance to the FAIR principles (Wilkinson et al., 2016), modularity, and practical deployability, we define a set of core design requirements that guide both the prototype architecture (Section 3) and its implemented functionalities (Section 4).

The Minimal Metadata Set (MNMS) concept, proposed by Moresis et al. (2024) for nonclinical in vivo data, was adapted and refined to meet the specific requirements of Home-Cage Monitoring (HCM). The metadata elements required to describe HCM datasets were derived to represent the entities and relationships that structure HCM experiments (Bains et al., 2025). This HCM-tailored metadata set was operationalized using Metadatapp to provide an interface for researchers to structure raw, proprietary HCM outputs into the MNMS-compliant JSON-LD manifests required for repository ingestion and validation. This approach, which is more directly based on the Home-Cage Monitoring Ontology (HCMO), explicitly captures key experimental components, including subjects, environment, hardware, software, and data outputs, and enforces consistency across datasets. The resulting conceptual model forms the basis of HCMO (Huzard, 2026), which is currently being developed as a community-driven semantic reference for HCM metadata.

Based on this framework, the prototype design is driven by the following eight requirements (R#) described in Table 1.

	Category	**Requirements details and specific rules**
R1	Multimodal linkage	The repository must support explicit linkage and temporal alignment across heterogeneous HCM data streams (e.g., video, audio, EMF, ECG, ...), by preserving per-stream timestamps, time reference/clock metadata, and alignment mappings (offset/drift) with associated quality/uncertainty over a valid time range.
R2	Machine-actionable validation	The system must enforce validation of a Minimal Metadata Set (MNMS) at ingestion to prevent metadata loss and ensure baseline data quality.
R3	Institutional sovereignty and federation	The architecture must reconcile global dataset discovery with institutional control through a zero-copy federation mechanism.
R4	API-first ingestion and retrieval	Programmatic interfaces must be provided to enable automated ingestion, access, and retrieval of data and metadata.
R5	Semantic interoperability	Semantic technologies (e.g. JSON-LD contexts and ontology hooks) must be used to ensure

		unambiguous data representation and cross-system interoperability.
R6	Storage-level access control	Access policies must be enforced directly at the physical storage layer (e.g. WAC/ACP) to provide legally robust and persistent security.
R7	Extensible adapters	The design must support modular integration of new data formats, storage backends, and mappings to domain or regulatory standards (e.g. SEND).
R8	Attribution and provenance	The system must capture provenance information (e.g. PROV-O) and support contributor attribution mechanisms (e.g. ORCID/CRediT) to incentivise data sharing.

Table 1. Core design requirements (noted R#) for the FAIR HCM federated repository prototype.

2.2 FAIR-Aligned Design Commitment

Across all architectural layers, the proposed repository framework aligns with the FAIR principles (Wilkinson et al., 2016) through explicit technical design choices.

Findable. Datasets are associated with persistent identifiers (e.g. DOIs, ROR, ORCID), and rich, structured metadata conforming to standardised schemas are exposed through machine-readable catalogues. These catalogues support indexing by a global discovery portal, enabling dataset search based on experimental, biological, and behavioural criteria.

Accessible. Data and metadata are made available through standard HTTP/HTTPS protocols. Authentication and authorisation mechanisms (e.g. OAuth 2.0, role-based access control) regulate access in accordance with data owner policies, while programmatic APIs enable automated access for downstream analysis workflows.

Interoperable. The framework supports common data representations (e.g. JSON, JSON-LD, CSV, RDF, OWL) and relies on semantic technologies—including ontologies, RDF graphs, and queryable relationships—to enable integration across tools and platforms.

Reusable. Datasets are accompanied by comprehensive metadata describing experimental design, methods, and provenance, as well as explicit licensing information. Data dictionaries, codebooks, and the Contributor Role Taxonomy (CRediT) linked to ORCID identifiers support correct interpretation, attribution, and reuse.

Together, these requirements and FAIR-aligned commitments define the design space for the prototype architecture described in the following section.

3. Prototype Architecture: A Decoupled, Federated Framework

The prototype adopts a decoupled, federated architecture that separates a local **Data Plane** (storage and access control) from a **Control Plane** (validation, governance, and discovery). This design reconciles institutional data sovereignty with global findability by enabling "zero-copy" federation: only standardised metadata are harvested centrally, while raw data remain under local control.

3.1. The Data Plane: Solid Pods

At the foundation of each Local Node lies a Solid Server (Social Linked Data). Solid provides the storage layer (Capadisi et al., 2020), treating data not as opaque blobs in a database, but as resources in a structured hierarchy of Linked Data Platform (LDP) containers (Speicher et al., 2015).

- Decentralized Storage: Each HCM dataset (metadata manifest plus multimodal assets such as time-series files and video objects) is stored in a dedicated hierarchy of LDP containers (e.g., /data/{year}/{dataset-id}/), enabling resource-level metadata attachment and stable identifiers.
- Access Control: Security is implemented via the W3C-recommended Web Access Control (WAC) or the newer Access Control Policy (ACP) specifications. This mechanism ensures that access to the underlying data files is governed by a decentralized identity and authentication layer (WebID/OIDC), making the access policy legally robust and persistent regardless of the overlying application layer.
- Data Integrity and Persistence: The use of standard, non-proprietary protocols (LDP) and decentralized identifiers guarantees the long-term persistence and machine-readability of the data resources.

3.2. The Control Plane: The Node Control Service

A Node Control Service (NCS), implemented using a mature open-source framework such as API Platform, developed by Les-Tilleuls.coop and based on the Symfony framework (Dunglas, 2025), functions as the Smart Gateway and validation layer for the Solid Data Plane. This decoupling provides a rigid, yet user-friendly, API abstraction over the complexity of LDP transactions.

- Ingestion & Validation: When a researcher submits a dataset, the Control Service accepts standard JSON-LD payloads (Manu Sporny et al., 2020). It validates the submission against the Minimal Metadata Set (MNMS) through two complementary layers: JSON Schema for structural and type correctness, and SHACL shapes for semantic validation against domain ontologies (e.g., HCMO). JSON Schema runs first in-process; SHACL validation is delegated to a dedicated sidecar service exposing a REST API (Knublauch & Kontokostas, 2017). Crucially, this validation occurs before

data is persisted to the Pod, preventing "metadata swamp" issues common in loose storage systems.

- LDP Orchestration: A state processor orchestrates persistence: after validation, the dataset manifest is written to the Solid Pod via the Solid Protocol (LDP), and the local query index is updated asynchronously. Container creation, URI minting (via Slug headers), and management of auxiliary resources (such as .acl files) are handled by the Pod interaction layer..
- Semantic Context Injection: To ensure semantic interoperability, the service dynamically injects W3C-compliant JSON-LD `@context` mappings (DCAT, Dublin Core Terms, PROV-O, schema.org) into the serialized response. Domain-specific vocabulary mappings are merged from a schema registry, ensuring that domain metadata terms resolve to the correct ontology IRIs (e.g., HCMO for HCM systems (Huzard, 2026), SOSA for sensor data (Haller et al., 2017)). This guarantees that data leaving the node is always semantically unambiguous.

3.3. Federation Strategy: The Catalog Endpoint

To achieve global findability without centralizing heavy data, each Node Control Service exposes a specialized federation endpoint: /catalog.jsonld.

- DCAT Compliance: The /catalog.jsonld Endpoint dynamically generates a Data Catalog Vocabulary (DCAT) representation of all public datasets stored in the local Solid Pods (Albertoni et al., 2024).
- Harvesting: A Global Portal crawls these lightweight catalog endpoints periodically. It aggregates the metadata into a central search index (e.g., Elasticsearch), allowing users to query across all institutions instantly.
- Zero-Copy Federation: The Global Portal stores only the dataset metadata—specifically the metadata.jsonld file that forms the basis of the catalogue—and a link to the original data object. . When a user requests access, they are redirected to the hosting institution's Node Control Service, where local access policies (WAC/ACP) are enforced. This architecture ensures Zero-Copy Federation: the raw data always remains at the originating institution and never resides on the central server.

3.4. Interoperability Plugins

A plugin architecture enables integration with existing research infrastructure:

- Storage Adapters: Connectors for various storage backends (institutional servers, cloud services, general repositories) allow flexibility in data management while maintaining the Solid interface.
- Format Converters: Tools to transform data between common formats (e.g., CSV to JSON) ensure interoperability with analysis software and other repositories.
- Standards Integration: Mappings to regulatory standards like SEND facilitate the translational pathway from discovery to application.

- Analysis Pipeline Connections: APIs that enable seamless integration with automated analysis tools and machine-learning frameworks, allowing external tools to access and process data from the repository.

The architecture is designed to support modularity through a plugin system, facilitating integration with diverse research and regulatory environments, as described in table 2.

Component	Functionality	Technical Detail
Standards Integration	Facilitates regulatory and translational research alignment.	Mappings to regulatory standards like SEND (Standard for Exchange of Nonclinical Data) and mappings to BioPortal ontologies.
data ingestion / preprocessing pipelines	Ensures compatibility with existing analysis tools and data-processing pipelines.	Tools that transform common HCM data formats (e.g., proprietary sensor logs, CSV) into standardized JSON or RDF models used by downstream analysis workflows.
Analysis Pipeline APIs	Enables automated, large-scale data processing.	Dedicated RESTful APIs for external tools (e.g., machine learning frameworks, cloud computing platforms) to programmatically query and retrieve data from the Solid Pods.
Storage Adapters	Accommodates varied institutional infrastructure.	Connectors that allow the Solid/NCS stack to interface with different physical backends (e.g., institutional NFS, Amazon S3, Google Cloud Storage) while maintaining a consistent Solid Pod interface.

Table 2. Interoperability components and their status.

In the current prototype, storage adapter support is implemented and demonstrated with Google Drive via rclone FUSE mounting (https://rclone.org/). Planned extensions include S3, NFS, format converters, SEND mappings, and analysis pipeline APIs. This architecture ensures the system is technically sustainable (leveraging mature open-source frameworks), standard-compliant (W3C LDP, JSON-LD, OIDC), and legally robust (data never leaves the institution without explicit authorization).

The prototype uses PHP 8.4 with the Symfony framework and API Platform for JSON-LD/Hydra/OpenAPI exposition. The Data Plane runs Community Solid Server. External storage is FUSE-mounted into the Pod filesystem via rclone, and the full stack is containerized for single-command deployment.

4. Prototype Functionalities

Section 3 defined the architecture; this section states what the prototype is designed to do in operational terms. The capabilities below are framed to match the R1–R8 requirements, described in table 1, and to be testable at the level of system behaviour: what goes in, what is checked, what becomes discoverable, and what remains protected.

4.1 Validated ingestion (metadata quality gate)

Dataset submission is handled through the Control Plane as JSON-LD payloads. Before any data are persisted, the Node Control Service performs two-layer validation against the MNMS: JSON Schema enforces structural correctness (required fields, types, enumerations), while SHACL shapes enforce semantic constraints (ontology alignment, controlled vocabulary membership, graph-level relationships). Both layers run before persistence—invalid or incomplete submissions are rejected at ingestion, preventing mandatory metadata from being silently lost into storage.

4.2 Dataset packaging as a first-class object (multimodal linkage)

Each dataset is treated as a package that includes a manifest plus multimodal assets (e.g. time-series files and video objects). Resources are organised within an LDP container hierarchy with stable internal identifiers. The manifest includes explicit linkage between assets and defines fields for temporal alignment metadata between modalities. These alignment descriptors may be provided by the data producer or generated during preprocessing when such information can be inferred from the raw data.

4.3 Storage-level identity and access control

Access policies are enforced at the Data Plane using WAC/ACP and WebID/OIDC. The Control Plane provides an API-facing abstraction for configuring dataset visibility without shifting enforcement away from storage. This is intended to keep permissions persistent and legally robust even if the application layer changes.

4.4 Federation-ready discovery without central custody

Each node generates a DCAT representation of public datasets at /catalog.jsonld. A harvester can index these catalogues centrally to support cross-node search, while leaving raw data in place. When access is requested, clients are directed back to the hosting node where local policies are enforced (zero-copy federation).

4.5 Interoperability surfaces via adapters

Interoperability is approached through canonical JSON-LD and a plugin layer for converters and mappings. The prototype is designed to support format conversion (e.g., CSV to JSON/RDF), storage adapters, and standard mappings as modular components rather than hard-coded, one-off integrations.

4.6 Provenance and attribution hooks

The data model includes provenance concepts (e.g., PROV-O) and contributor attribution mechanisms (e.g., ORCID/CRediT) at the metadata level. The prototype is designed so that dataset creation and transformation steps can be represented explicitly, supporting traceability and citable contribution records.

5. Prototype implementation and demonstration workflows

The prototype is a concrete demonstration of the architecture defined in Section 3, translating the R1–R8 requirements into actionable, deployed components. It moves beyond a theoretical framework to provide verifiable, working examples of its core functions: data sovereignty, machine-actionable validation, and federated discovery.

5.1 Deployment Model (The Local Node)

A single federated instance, or "Local Node," is composed of two decoupled components: a Data Plane and a Control Plane (see Figure 1). The Data Plane is implemented by a Solid Server managing the physical storage for all HCM datasets. The Control Plane is the Node Control Service (NCS), built on an API Platform framework. The NCS acts as the gateway, orchestrating all data ingestion, validation, and metadata generation, while the Solid Pods enforce access control at the storage layer. This separation is the foundation of institutional data sovereignty: raw data files remain on institutional infrastructure (FUSE-mounted into the Pod), metadata manifests are persisted to the Pod as JSON-LD, and all access is governed by WAC/ACP. The NCS maintains a local query-optimized index for efficient filtering, search, and catalog generation; the Solid Pod remains the authoritative store from which this index can be reconstructed.

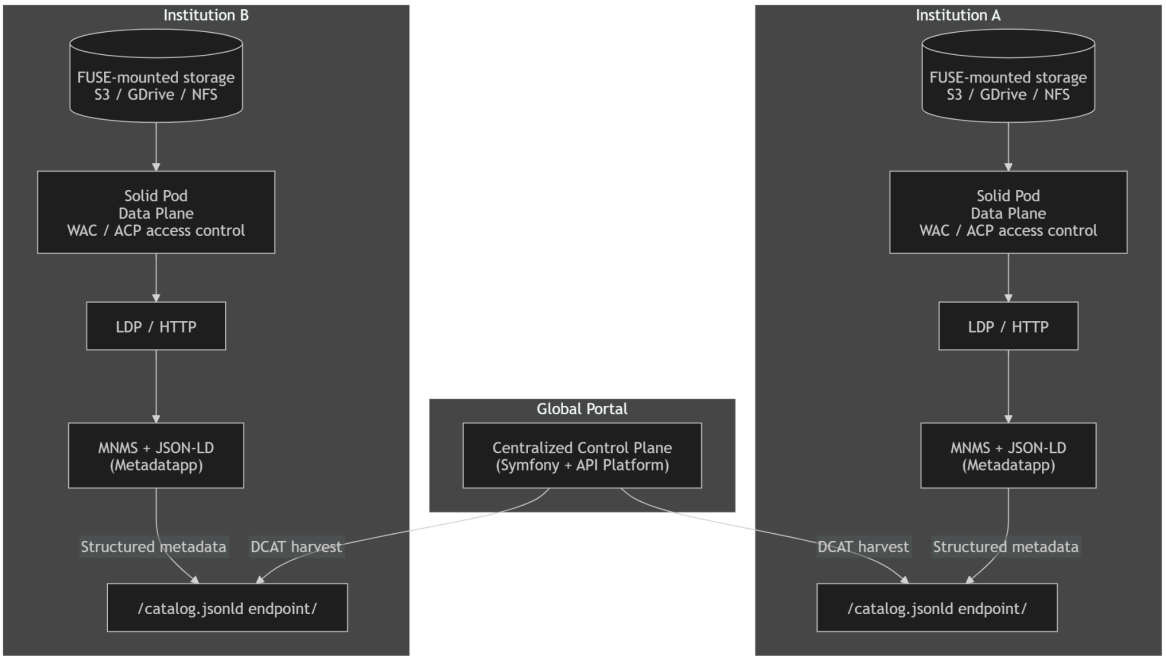

Figure 1. Centralized control plane with federated data plane for metadata-driven data sharing. A Global Portal, implemented with Symfony and API Platform, acts as a centralized control plane that harvests DCAT catalogs (/catalog.jsonld) from distributed institutions. Within each institution, raw data are stored locally via FUSE-mounted storage backends (e.g., S3, GDrive, NFS) and exposed through Solid Pods forming the data plane, with access governed by WAC/ACP policies. Data are accessed via LDP/HTTP and transformed into structured, MNMS-compliant JSON-LD metadata by a Metadatapp layer, which serves as a semantic bridge between data and APIs. These metadata are exposed through institutional catalog endpoints and harvested by the Global Portal, enabling federated discovery without moving underlying data (zero-copy), while preserving institutional data sovereignty.

5.2 Data Model (Dataset Manifests and Assets)

Each HCM dataset is managed as a first-class object, consistent with R1 (Multimodal Linkage). A dataset is packaged as a JSON-LD manifest stored in an LDP container on the Solid Pod, alongside references to multimodal assets (various sensor datasets, time-series files, video objects) served at stable Pod URLs. The manifest contains DCAT-compliant core fields (title, description, keywords, distributions) and a flexible domain metadata section validated against a registered JSON Schema and SHACL shapes. The NCS indexes this metadata locally for query performance and catalog generation. This package includes a canonical JSON-LD Dataset Manifest (the machine-readable metadata file) and the associated multimodal assets (e.g., time-series files and video objects). The manifest includes explicit fields to link the assets. Temporal alignment metadata (e.g., start time, synchronisation offset) can be attached at ingestion or enriched post-hoc via the API when asset analysis has been performed.

5.3 Validation Implementation (Schemas and Shapes)

To satisfy R2 (Machine-Actionable Validation), the Node Control Service enforces a metadata quality gate. When a researcher submits a dataset manifest, the NCS validates it through two sequential layers. First, a JSON Schema validator checks structural correctness: required properties, data types, and enumerated values. Second, the domain metadata is converted to JSON-LD (by injecting the domain vocabulary context) and submitted to a SHACL validation sidecar, which enforces semantic constraints—ontology-defined controlled vocabularies, graph-level nesting rules, and RDF datatype correctness. If either layer rejects the payload, the submission fails with machine-readable violation details. This two-gate validation is performed prior to persistence, ensuring the mandatory MNMS is complete and semantically valid, preventing the common "metadata swamp" issue (Figure 2). New scientific domains add their own JSON Schema and SHACL shapes files without application code changes.

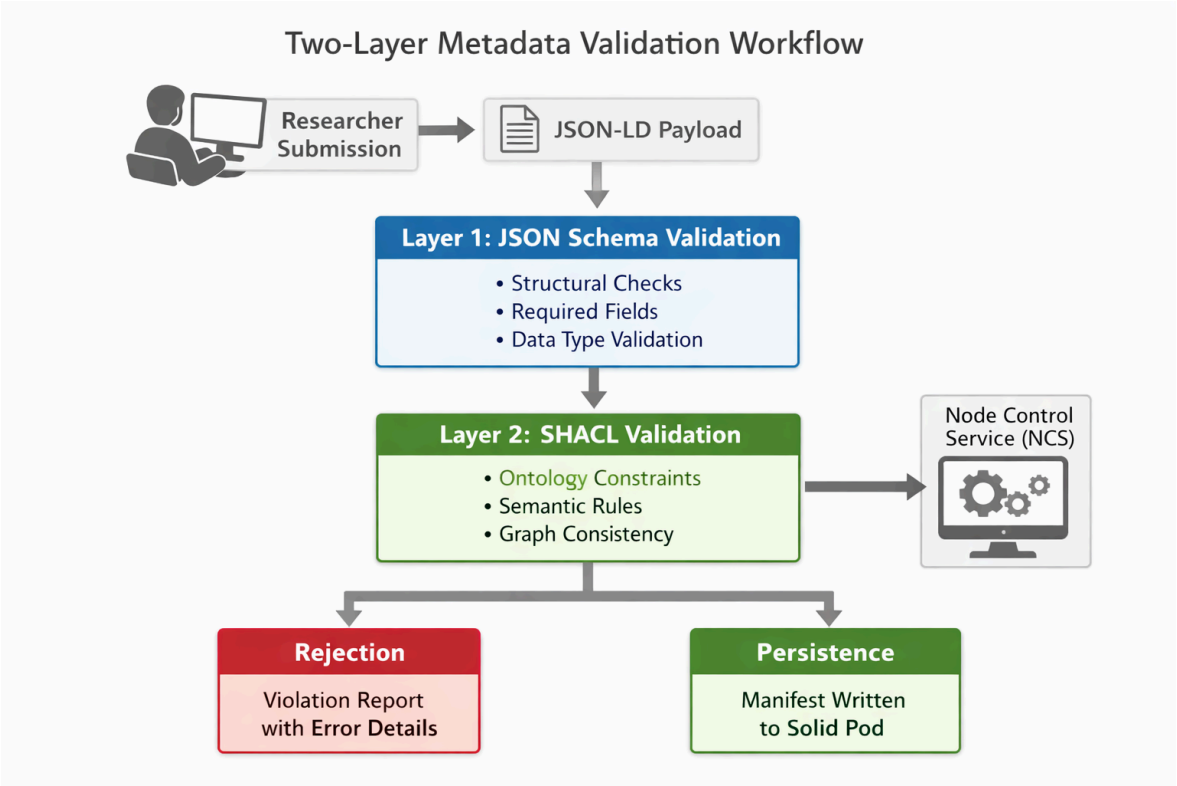

Figure 2. Two-Layer Metadata Validation Workflow. The flow illustrates the mandatory quality gate enforced by the Node Control Service (NCS) upon dataset manifest submission. Incoming JSON-LD payloads must pass two sequential layers: Layer 1 (JSON Schema) validates structural correctness, required fields, and data types; subsequently, Layer 2 (SHACL shapes) enforces semantic constraints, ontology alignment (HCMO), and graph-level integrity. Only after passing both layers is the manifest persisted to the Solid Data Plane, ensuring the integrity of the Minimal Metadata Set (MNMS).

5.4 Federation Demonstration (Catalog Generation and Harvest)

Global discoverability without sacrificing local control (R3) is demonstrated through Zero-Copy Federation. Each Local Node dynamically generates and exposes a standardized DCAT-compliant catalogue at the /catalog.jsonld endpoint. This lightweight endpoint contains only the structured metadata and links to the original resources, with HTTP caching headers for efficient periodic harvesting.. A Global Portal (the federation harvester) periodically crawls this endpoint, aggregating only the metadata into a central search index, ensuring the raw data never resides on the central server.

5.5 Access Control Demonstration (Dataset Visibility)

The prototype confirms R6 (Storage-Level Access Control) by demonstrating three dataset visibility modes (private, consortium, and public). The Control Plane records these settings and associates them with WebID/OIDC identity metadata. In the current prototype, these visibility settings are functional at the metadata layer and exposed for catalog generation. Full, storage-level enforcement of access policies using Web Access Control (WAC) or Access Control Policy (ACP) is a planned extension that will fully integrate with the Solid Pods, ensuring the access policy is legally robust and enforced directly at the physical data resource layer.

Lowering the barrier to contribution matters as much as the architecture. The adoption path described here focuses on incremental participation: teams can deploy locally to organise data first, then expose selected datasets via catalog metadata as governance and confidence mature. This is supported by layered metadata (mandatory, recommended, optional), controlled vocabulary with room for local extensions, and contributor recognition (CRediT/ORCID) that makes curation visible and citable.

6. Sociotechnical Considerations for Adoption

A federated repository will not succeed on software alone. HCM labs differ widely in resources, workflows, and tolerance for overhead. The design therefore assumes incremental uptake, where local utility comes first and federation comes later.

6.1 Flexibility and Inclusiveness

The system must accept heterogeneous studies without collapsing into free text. A layered metadata approach (mandatory, recommended, optional) helps keep the minimum stable while allowing richer description where feasible. For behavioural terminology, a controlled vocabulary with room for local extensions

supports comparability without blocking legitimate, study-specific definitions. This is aligned with the ongoing development of the Home-Cage Monitoring Ontology (HCMO) draft.

6.2 Incentives for Sharing

Curation competes with experiments for time, and the academic reward structure often ignores it. The repository therefore treats attribution as part of the metadata layer: contributor roles can be recorded (CRediT/ORCID), and datasets can be made citable via persistent identifiers (e.g., DOI). The point is not publicity; it is traceable credit and clearer provenance.

6.3 Bridging Local and Global Initiatives:

Institutions need control over storage location, access permissions, and compliance constraints. The federated model supports that by keeping custody local while making discovery global through lightweight catalogue metadata. Importantly, participation is not all-or-nothing: a lab can start with private local deployment, then expose datasets selectively to the global index when policies and confidence allow.

7. Discussion and Future Work

This prototype targets a specific failure mode in HCM research: data exist, but their structure, linkage, and governance prevent reliable reuse across sites. The central design choice—decoupling custody (Data Plane) from validation and discovery (Control Plane)—is meant to address that failure mode while respecting institutional sovereignty.

Near-term work is largely about hardening and community fit: (i) tightening the MNMS validation rules as experience accumulates, (ii) refining the dataset manifest to cover common edge cases in multimodal alignment, (iii) expanding adapter coverage for real-world HCM exports and storage backends, and (iv) operationalising governance patterns for consortium sharing (identity, permissions, review of public release).

The more ambitious goal is a network effect: multiple nodes exposing consistent, machine-readable catalogues that can be indexed without pulling data into a central warehouse. If that works, it becomes realistic to discover comparable datasets across institutions, request access under local policy, and reuse data with sufficient context to avoid accidental misinterpretation. The synergy between Metadatapp (specialized in domain-specific data standardization, like MNMS) and Les-Tilleuls.coop (experts in API Platform and Solid infrastructure deployment) offers a blueprint for building and hosting production-grade, federated FAIR data systems for various big projects, not limited to HCM data.

Data availability

The authors declare they used only mock data in the creation of this article. Therefore, no data must be made available.

Code availability

The prototype source code is available upon request. The repository (closed on GitHub) includes the Node Control Service, Docker Compose configuration, domain JSON Schemas, SHACL shapes, fixture data, and functional tests.

License

proprietary

Acknowledgements / Fundings

Damien is greatly thankful to the E.P.V team for eternal support.

Contributions (CREDIT)

Conceptualization
Damien, Antoine, Kevin
Data curation
Damien, Antoine
Formal analysis
Damien, Antoine, Kevin
Funding acquisition
Damien, Antoine, Kevin
Methodology
Damien, Antoine, Kevin
Project administration
Damien
Software
Antoine
Supervision
Damien, Antoine, Kevin
Validation
Antoine, Damien
Visualization
Damien, Antoine
Writing – original draft
Damien, Antoine
Writing – review & editing
Damien, Antoine, Kevin

Ethics declarations

Competing interests

Damien Huzard is the owner of NeuroNautix (registered as sole proprietorship in France) and the creator of Metadatapp, a metadata management solution. The work aligns with their scope of activity and may indirectly increase their visibility. Antoine Bluchet and Kevin Dunglas are respectively, CTO and CEO, of Les-Tilleuls.coop an IT consulting company specialized in PHP and Symfony/Laravel development, and created the API Platform framework.

References

Armin Haller, Krzysztof Janowicz, Simon Cox, Danh Le Phuoc, Kerry Taylor, & Maxime Lefrançois. (2017). *Semantic Sensor Network Ontology (SOSA)* [W3C Recommendation]. https://www.w3.org/TR/vocab-ssn/

Bains, R., Huzard, D., McCutcheon, J., Boguszewski, P., Virag, D., Restivo, L., Lewejohann, L., Ashby, M., Rozman, J., Marion, R., Gates, H., Forrest, H., Kalliokoski, O., & Girard, B. (2025). *Too big to lose—A FAIR repository for biomedical data derived from home-cage monitoring*. OSF. https://doi.org/10.31219/osf.io/fsg83_v1

Dunglas, K. (2025). *API PLATFORM* (Version v4.1.0) [TS, PHP]. Les-Tilleuls.coop. https://api-platform.com/

Holger Knublauch & Dimitris Kontokostas. (2017). *Shapes Constraint Language (SHACL)* [W3C Recommendation]. https://www.w3.org/TR/shacl/

Huzard, D. (2026). *The Home-Cage Monitoring Ontology (HCMO)* [Ontology]. https://doi.org/10.5281/zenodo.18925285

Manu Sporny, Dave Longley, Gregg Kellogg, Markus Lanthaler, Pierre-Antoine Champin, & Niklas Lindström. (2020). *JSON-LD 1.1* (W3C Recommendation 1.1). https://www.w3.org/TR/json-ld/

Riccardo Albertoni, David Browning, Simon J D Cox, & Alejandra Gonzalez Beltran. (2024). *Data Catalog Vocabulary (DCAT)* [W3C Recommendation]. https://www.w3.org/TR/vocab-dcat-3/

Sarven Capadisi, Tim Berners-Lee, & Kjetil Kjernsmo. (2020). *Solid Protocol* [Specification]. W3C Solid Community Group. https://solidproject.org/TR/2024/protocol-20240512

Steve Speicher, John Arwe, & Ashok Malhotra. (2015). *Linked Data Platform 1.0* (Recommendation Http://www.w3.org/TR/2015/REC-Ldp-20150226/). W3C. http://www.w3.org/TR/ldp/

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 160018. https://doi.org/10.1038/sdata.2016.18